Cost-Accounted Rho Calculus

A Spectral Decomposition of Phlogiston

L. Gregory Meredith*

May 2026

Abstract

We present a revision of the rho calculus in which cost accounting is not an external extension to be translated back into the core language, but is *folded directly into the grammar and operational semantics* of the calculus itself. Signatures become a species of name alongside quoted processes; token stacks become first-class terms; and the for-comprehension and send operators range over *signed terms* rather than bare processes. The result is a calculus in which every communicating process must engage the cost layer — embodying the principle of *valuable friction* at the syntactic level. The signature grammar is parameterized by a type variable G representing the cryptographic backend, making the calculus generic over signature schemes.

We identify the for-comprehension as the transaction primitive of the calculus: the binding specification is the transaction, the substitution is the witness, and application to the continuation is the state update. Individual transactions are atomic by construction. Multi-step transactions are made atomic by a decidable static analysis at deployment-acceptance time: a linear resource proof verifying that every for-comprehension in the deployment's call graph is funded before any step executes.

Because the rho calculus is the formal core of Rholang, this revised calculus serves as the design specification for rearchitecting phlogiston accounting in Rholang 1.4 (MeTTaIL) and the F1R3FLY platform.

Contents

*F1R3FLY.io; F1R3FLY Industries. lgreg.meredith@f1r3fly.io

1 Introduction

The rho calculus [1] is a reflective higher-order process calculus: channels are the quoted codes of processes, and processes may be recovered from channels by dequotation. As the formal core of Rholang [3], the rho calculus underpins the computation and storage model of the F1R3FLY platform.

Rholang's cost-accounting mechanism — the *phlogiston* (phlo) system — gates every deploy behind a token balance associated with a digital signature. In previous work, this mechanism was specified as an *extension* of the rho calculus: additional syntax for signatures and tokens, and additional rewrite rules for gated communication. One could then *translate* the extension back into the pure rho calculus, demonstrating that cost accounting is expressible within the language.

This paper takes the next step. Rather than treating cost accounting as an external layer and then recovering it via translation, we *fold it directly into the calculus*. The key design moves are:

(i) **Signatures are names.** The name sort is extended so that $x, y ::= @T \mid s$, where s ranges over signatures. Signatures and quoted processes coexist as first-class channel names.

(ii) **Two axes of reflection.** Alongside the structural quoting operator $@T$ (the code of a signed term used as a channel), a *cryptographic quoting* operator $\#P$ (the hash of a process) appears in the signature grammar. This gives the calculus two axes of reflection: one for communication, one for authentication.

(iii) **Signed terms pervade the syntax.** The for-comprehension and send take *signed terms* T, U as their body and payload respectively, rather than bare processes. One cannot write a communicating process without engaging the cost layer.

(iv) **Token stacks are first-class terms.** Token stacks $S ::= () \mid s : S$ are included in the signed-term grammar, so they participate in reduction on equal footing with signed processes.

(v) **Parametric polymorphism.** Parallel composition is overloaded: $P \mid Q$ composes processes, $T \mid U$ composes signed terms. The operator is parametrically polymorphic in the sort it composes.

(vi) **Generic cryptography.** The signature grammar is parameterized by a type variable G representing the ground signature scheme (Ed25519, secp256k1, etc.), making the calculus independent of any particular cryptographic backend.

Because the rho calculus *is* the core of Rholang, this revised calculus is not a theoretical curiosity — it is the design specification for rearchitecting phlogiston accounting in Rholang 1.4 (MeTTaIL) and the F1R3FLY platform.

2 Motivation: From Run-to-Completion to Concurrent Acceptance

The redesign presented in this paper is motivated by a fundamental architectural limitation in the current F1R3FLY implementation, inherited from RChain.

2.1 The Run-to-Completion Model

In the current architecture, each deployment submitted to a F1R3Node is executed under *run-to-completion* semantics: the RSpace tuplespace engine accepts a deployment, executes it to termination (or until phlogiston is exhausted), commits the resulting state changes, and only then accepts the next deployment. This model introduces two related problems:

(i) **Implicit transaction boundaries.** The entire deployment is treated as a single, implicit transaction. There is no formal specification of what constitutes a transaction — the deployment boundary *is* the transaction boundary, by convention rather than by design. This conflates the unit of submission (the deployment) with the unit of atomicity (the transaction), making it impossible to express deployments that contain multiple independent transactions or to share transactional structure across deployments.

(ii) **Limited availability.** Because RSpace executes deployments sequentially, it is *locked* for the duration of each deployment's execution. No other deployment can be accepted, pattern-matched, or reduced while the current deployment is running. The throughput of the F1R3Node is bounded by the execution time of the slowest deployment in the queue. This is a severe bottleneck: a single expensive deployment blocks all other activity on the node.

2.2 The Merge Analysis Burden

To recover concurrency in the current architecture, F1R3FLY (following RChain) employs *merge analysis*: after multiple deployments have been executed independently (e.g., by different validators or in speculative parallel execution), their resulting states must be *merged* into a single consistent state. The merge analysis determines whether the independently computed states are compatible (no conflicting writes to the same tuplespace entries) and, if so, how to combine them.

Merge analysis is expensive, complex, and a persistent source of bugs. It requires tracking read and write sets for every deployment, computing conflict graphs, and resolving or rejecting conflicting state transitions. Moreover, the analysis is *post-hoc*: deployments are executed speculatively, and conflicts are detected only after the work has been done. Wasted computation is the norm, not the exception.

2.3 The New Model: Acceptance by Linear Proof

The cost-accounted rho calculus eliminates both problems by replacing run-to-completion with *concurrent acceptance gated by linear resource proofs*.

Under the new model:

(i) **Explicit transaction boundaries.** The for-comprehension is the transaction primitive (Section 7.1). Transaction boundaries are determined by the syntactic structure of signed terms, not by deployment boundaries. A deployment may contain multiple transactions; multiple deployments may participate in a single financial transaction (via shared signature channels).

(ii) **Concurrent acceptance.** RSpace can accept deployments *at any time*, provided each deployment carries a linear resource proof (Section 7.5) demonstrating that sufficient tokens exist to complete all of its transactions. Multiple deployments can be active simultaneously, each drawing from its own committed token supply. RSpace is never locked by a single deployment.

(iii) **Throughput.** Because acceptance is gated by a static check (linear in the AST size) rather than by sequential execution, the F1R3Node can accept and begin executing deployments in parallel. The throughput bottleneck shifts from "one deployment at a time" to "as many deployments as there are non-conflicting token supplies" — a dramatic improvement.

(iv) **Merge via signatures and cost accounting.** The need for post-hoc merge analysis is substantially reduced. In the cost-accounted calculus, conflicts are detected *at acceptance time* through the linear resource proof: two deployments that compete for the same tokens are two proof obligations competing for the same linear hypotheses, and at most one can succeed (Section 7.7). Merging is no longer a separate analysis pass over execution traces — it is a consequence of the signature and cost-accounting structure of the calculus itself.

Specifically, deployments signed by *different* signatures draw from *disjoint* token pools and cannot conflict. They may be executed in parallel and their results composed without merge analysis. Deployments signed by the *same* signature (or overlapping compound signatures) compete for the same token pool and are serialized by the linear proof — again without merge analysis. The only case requiring attention is deployments that interact via shared data channels, and even here the cost-accounting structure provides a natural serialization order.

Remark 2.1 (From blocking to budgeting)**.** *The architectural shift can be summarized in a single sentence: the current model asks "is the tuplespace free?" (blocking); the new model asks "is this deployment funded?" (budgeting). The first question serializes all activity. The second question can be answered independently for each deployment, enabling concurrency proportional to the diversity of the token supply.*

2.4 New Capability: Multi-Signature Execution

Beyond streamlining execution and increasing throughput, the cost-accounted calculus introduces a feature absent from the current architecture: *n-party multi-signature execution.*

The compound signature $s_1 * s_2 * \cdots * s_n$ in the signed term $\{P\}_{s_1 * \cdots * s_n}$ requires that *all* n parties explicitly sign off on the execution of P. The rewrite rules enforce this structurally: the process cannot fire unless tokens from every component signature are present. There is no separate multi-sig verification layer, no threshold-signature scheme, no off-chain coordination protocol. Multi-sig is a consequence of the grammar.

Programmable tokenized capabilities. The interaction between compound signatures and token stacks yields a rich *object-capability* model with programmable amplification and attenuation:

Attenuation.
A token stack limits what a capability can do. A process signed by s with a two-token stack $s : s : ()$ can perform at most two funded actions. By controlling the depth of the token stack granted to a subprocess, a parent process *attenuates* the subprocess's authority. This is token-metered capability attenuation: the child can do no more than the parent has budgeted.

Amplification.
Composing signatures via $*$ *amplifies* authority. A process signed by $s_1 * s_2$ can perform actions that neither s_1 nor s_2 could authorize alone. For example, a funds-transfer contract might require $\{transfer\}_{s_{sender} * s_{compliance}}$, meaning the transfer executes only if both the sender and the compliance officer have signed. Neither party alone can move funds; their composed authority can.

Programmable delegation.
Because signatures are names and token stacks are first-class terms, a process can *delegate*

authority by sending tokens on a signature channel. A manager process holding $s_{mgr} : S$ can send a portion of its stack to a subordinate process, granting the subordinate a metered budget of the manager's authority. When the subordinate exhausts its budget, it halts — no revocation protocol needed.

Just-in-time funding.
Because token stacks are first-class terms, they can be *constructed and supplied dynamically* — not merely pre-allocated at deployment time. A contract can mint a token stack $s : s : ()$ and send it on a signature channel at the precise moment it is needed, funding a capability on an as-needed basis.

This enables natural *on-boarding* patterns: a platform contract grants a new user a small slush fund (a shallow token stack) to explore the system. The user's initial deployments are funded by this grant, letting them experiment without acquiring tokens of their own. Once the slush fund is exhausted, further execution requires the user to acquire tokens through the token economy — purchase, earn, or receive via delegation. The transition from subsidized exploration to self-funded participation is not a policy check external to the calculus; it is a *resource exhaustion event* within it.

This capability model is *intrinsic* to the calculus: amplification, attenuation, delegation, and just-in-time funding are not library features or design patterns but direct consequences of the signature algebra and token-stack semantics. The F1R3FLY platform's token-attenuated object-capability model for agentic AI governance [5] is a direct application of this structure.

3 The Cost-Accounted Rho Calculus

3.1 Syntax

The calculus has four mutually defined syntactic categories: *processes*, *names*, *signed terms*, and *signatures*.

Definition 3.1 (Processes and names)**.**

$$P, Q \ ::= \ \mathbf{0} \ \mid \ \mathbf{for}(y \leftarrow x)\, T \ \mid \ x!(U) \ \mid \ P \,|\, Q \ \mid \ *x \tag{1}$$

$$x, y \ ::= \ @T \ \mid \ s \tag{2}$$

The intended readings carry over from the pure rho calculus, with one critical change: the for-comprehension $\mathbf{for}(y \leftarrow x)\, T$ has a *signed term* T as its continuation, and the send $x!(U)$ carries a signed term U as its payload.

Definition 3.2 (Signed terms and token stacks)**.**

$$T, U \ ::= \ \{P\}_s \ \mid \ T \,|\, U \ \mid \ S \tag{3}$$

$$S \ ::= \ () \ \mid \ s : S \tag{4}$$

A signed term is one of:

$\{P\}_s$
: A process P signed by signature s. The braces delimit the scope of the signature.

$T \,|\, U$
: Parallel composition of signed terms — the same $|$ operator as at the process level, lifted to signed terms.

S A token stack, representing a phlogiston balance.

A token stack $s_1 : s_2 : \cdots : s_n : ()$ is read as a sequence of signature-indexed balance layers. The empty stack $()$ represents a depleted balance.

Definition 3.3 (Signatures, parameterized by G)**.**

$$s(G) \ ::= \ g \ \mid \ \#P \ \mid \ s * s \tag{5}$$

where $g \in G$ is a ground signature *drawn from the cryptographic backend G.*

The three forms of signature are:

g A *ground signature*: an opaque value from the cryptographic backend G (e.g., an Ed25519 public key, a secp256k1 key hash). The calculus is parametrically polymorphic in G.

$\#P$ *Cryptographic quoting*: the hash of a process P, yielding a signature. This is the authentication-axis analog of structural quoting $@P$.

$s_1 * s_2$
Signature composition: the compound authority of s_1 and s_2 acting jointly.

Remark 3.4 (Two axes of reflection)**.** *The calculus now has two quoting operators:*

Operator	***Axis***	***Purpose***
$@T$	*Structural*	*Signed term → channel name*
$\#P$	*Cryptographic*	*Process → signature*

Both are forms of reflection*: they take a computational entity and produce a* name *(a datum that can be communicated, compared, and used as an address). The difference is that $@T$ preserves full structural information — including the signature and cost provenance of T, and the signed term can be recovered via $*x$ — while $\#P$ is a one-way, collision-resistant digest, suitable for authentication but not for recovery.*

3.2 Parametric Polymorphism of Parallel Composition

The parallel composition operator $\mid$ appears at two levels:

1. $P \mid Q$ — parallel composition of processes.
2. $T \mid U$ — parallel composition of signed terms.

This is a parametrically polymorphic operator: the same symbol, the same algebraic structure (commutative monoid with appropriate identity), but ranging over different sorts. At the process level the identity is $\mathbf{0}$; at the signed-term level the identity is the empty stack $()$.

3.3 Free Names

The binding discipline carries over from the pure rho calculus: $\mathbf{for}(y \leftarrow x)\, T$ binds y. Free names are computed recursively over both processes and signed terms:

$$\begin{aligned}
\mathrm{FN}(\mathbf{0}) &= \emptyset \\
\mathrm{FN}(\mathbf{for}(y \leftarrow x)\, T) &= \big(\mathrm{FN}(T) \setminus \{y\}\big) \cup \{x\} \\
\mathrm{FN}(x!(U)) &= \mathrm{FN}(U) \cup \{x\} \\
\mathrm{FN}(P \mid Q) &= \mathrm{FN}(P) \cup \mathrm{FN}(Q) \\
\mathrm{FN}(*x) &= \{x\}
\end{aligned}$$

Extended to signed terms:

$$\begin{aligned}\mathrm{FN}(\{P\}_s) &= \mathrm{FN}(P) \cup \mathrm{FN}_s(s)\\ \mathrm{FN}(T \mid U) &= \mathrm{FN}(T) \cup \mathrm{FN}(U)\\ \mathrm{FN}(()) &= \emptyset\\ \mathrm{FN}(s : S) &= \mathrm{FN}_s(s) \cup \mathrm{FN}(S)\end{aligned}$$

where FN_s extracts names from signatures:

$$\begin{aligned}\mathrm{FN}_s(g) &= \emptyset\\ \mathrm{FN}_s(\#P) &= \mathrm{FN}(P)\\ \mathrm{FN}_s(s_1 * s_2) &= \mathrm{FN}_s(s_1) \cup \mathrm{FN}_s(s_2)\end{aligned}$$

3.4 Structural Equivalence

Structural equivalence $\equiv_S$ is the smallest equivalence relation on processes including α-equivalence and making $(P, |, \mathbf{0})$ a commutative monoid:

$$\begin{aligned}P \mid \mathbf{0} &\equiv_S P\\ P \mid Q &\equiv_S Q \mid P\\ (P \mid Q) \mid R &\equiv_S P \mid (Q \mid R)\end{aligned}$$

Likewise, at the signed-term level, $(T, |, ())$ forms a commutative monoid under $\equiv_S$:

$$\begin{aligned}T \mid () &\equiv_S T\\ T \mid U &\equiv_S U \mid T\\ (T \mid U) \mid V &\equiv_S T \mid (U \mid V)\end{aligned}$$

3.5 Substitution

Substitution $T\{@U/y\}$ is the standard capture-avoiding substitution of $@U$ for y in T, extended to signed terms. Following the pure rho calculus, the only non-trivial case is dequotation:

$$(*x)\{@U/y\} \;=\; \begin{cases} U & \text{if } y \equiv_N x,\\ *x & \text{otherwise,}\end{cases}$$

where $\equiv_N$ denotes name equality.

Remark 3.5 (Signed-term substitution)**.** *Because the name sort is $x, y \;::=\; @T \mid s$, the substitution $T\{@U/y\}$ naturally substitutes the quoted* signed term U*, not a bare process extracted from U. There is no unwrapping. This means the signature and cost information associated with U is* preserved through communication*: a signed term received on a channel retains its provenance.*

3.6 Rewrite Rules

The operational semantics consists of five rewrite rules, each a variation on the COMM rule of the pure rho calculus gated by token consumption. In every rule, a communication fires only when matching tokens are present, and the consumed tokens are released as residual terms.

Rule 1 — Single signature, whole redex:

$$\boxed{\{\mathbf{for}(y \leftarrow x)\, T \mid x!(U)\}_s \;\mid\; s : S \;\;\rightsquigarrow\;\; T\{@U/y\} \mid S} \tag{6}$$

The entire redex (input and output) is signed by s. A single layer of token is consumed; the stack remainder S is released.

Rule 2 — Compound signature, whole redex, split tokens:

$$\boxed{\{\mathbf{for}(y \leftarrow x)\, T \mid x!(U)\}_{s_1 * s_2} \;\mid\; s_1 : S \mid s_2 : S' \;\;\rightsquigarrow\;\; T\{@U/y\} \mid S \mid S'} \tag{7}$$

The redex is signed by the compound authority $s_1 * s_2$. Both component signatures must supply tokens; both remainders are released.

Rule 3 — Compound signature, whole redex, combined token:

$$\boxed{\{\mathbf{for}(y \leftarrow x)\, T \mid x!(U)\}_{s_1 * s_2} \;\mid\; (s_1 * s_2) : S \;\;\rightsquigarrow\;\; T\{@U/y\} \mid S} \tag{8}$$

The compound authority $s_1 * s_2$ holds a single combined token. One layer is consumed; the remainder S is released.

Rule 4 — Split processes, split tokens:

$$\boxed{\{\mathbf{for}(y \leftarrow x)\, T\}_{s_1} \;\mid\; \{x!(U)\}_{s_2} \;\mid\; s_1 : S \mid s_2 : S' \;\;\rightsquigarrow\;\; T\{@U/y\} \mid S \mid S'} \tag{9}$$

Input and output are signed separately. Each signer supplies their own token.

Rule 5 — Split processes, combined token:

$$\boxed{\{\mathbf{for}(y \leftarrow x)\, T\}_{s_1} \;\mid\; \{x!(U)\}_{s_2} \;\mid\; (s_1 * s_2) : S \;\;\rightsquigarrow\;\; T\{@U/y\} \mid S} \tag{10}$$

Input and output are signed separately, but the token is held jointly under the compound authority.

As in the pure rho calculus, these rules are closed under parallel context and structural equivalence:

$$\frac{P \rightsquigarrow P'}{P \mid Q \rightsquigarrow P' \mid Q} \quad (\textsc{Par}) \tag{11}$$

$$\frac{P \equiv_S P' \qquad P' \rightsquigarrow Q' \qquad Q' \equiv_S Q}{P \rightsquigarrow Q} \quad (\textsc{Struct}) \tag{12}$$

3.7 Summary of the Five Rules

The five rules can be organized along two axes:

	Split tokens	**Combined token**
Whole redex, single sig	Rule 1	Rule 1
Whole redex, compound sig	Rule 2	Rule 3
Split processes	Rule 4	Rule 5

Every rule follows the same pattern: *communication is gated by token consumption.* The variations arise from the granularity of signing (whole redex vs. split processes) and the granularity of token holding (split vs. combined).

3.8 Syntactic Sugar

Two abbreviations streamline the common case of signing a for-comprehension together with its continuation.

Definition 3.6 (Uniform signing)**.**

$$\{\mathbf{for}(y \leftarrow x)\, P\}_s \quad \triangleq \quad \{\mathbf{for}(y \leftarrow x)\, \{P\}_s\}_s \tag{13}$$

When a for-comprehension and its continuation are signed by the *same* signature s, the abbreviation on the left expands to two nested signed layers on the right. The outer $\{\cdot\}_s$ pays for the transaction (rendezvous and matching); the inner $\{\cdot\}_s$ pays for executing the continuation. This is the common case for single-party execution: both the communication and its effect are authorized and funded by the same signer.

Definition 3.7 (Linear transfer of authority ($\multimap$))**.**

$$\{\mathbf{for}(y \leftarrow x)\, P\}_{s_1 \multimap s_2} \quad \triangleq \quad \{\mathbf{for}(y \leftarrow x)\, \{P\}_{s_2}\}_{s_1} \tag{14}$$

The *lollipop* notation $\{\cdot\}_{s_1 \multimap s_2}$ denotes a *transfer of authority*: signature s_1 authorizes and funds the communication, but signature s_2 authorizes and funds the continuation. The $\multimap$ is deliberately chosen to evoke the linear implication of linear logic: s_1's resource is consumed to establish the communication, and s_2's resource funds the resulting computation.

Example 3.8 (Cross-party service invocation)**.** *A client c invokes a service operated by provider p. The client pays for the call; the provider pays for the execution:*

$$\{\mathbf{for}(\mathit{result} \leftarrow \mathit{service})\, \mathit{handleResult}\}_{c \multimap p} \quad = \quad \{\mathbf{for}(\mathit{result} \leftarrow \mathit{service})\, \{\mathit{handleResult}\}_p\}_c$$

The client's token funds the rendezvous with the service channel; the provider's token funds the execution of handleResult. Neither party can impose costs on the other beyond what the sugar makes explicit.

Remark 3.9 (Composability)**.** *Both sugar forms compose with compound signatures. For example, $\{\mathbf{for}(y \leftarrow x)\, P\}_{(s_1 * s_2) \multimap s_3}$ requires both s_1 and s_2 to authorize the communication, while s_3 alone funds the continuation. Chains of transfers are also expressible: nested for-comprehensions can each carry their own $\multimap$, threading authority through a pipeline of parties.*

4 Design Analysis

4.1 Signatures as Names: Eliminating the Translation Layer

In a previous formulation, the cost-accounted calculus was specified as an extension of the pure rho calculus, and a compositional translation $\mathcal{N}[\![\cdot]\!]$ mapped signatures to names, tokens to messages, and signed processes to fuel-gated wrappers. While that translation demonstrated expressibility, it introduced an indirection: the implementation had to maintain two representations and a mapping between them.

The present formulation eliminates this indirection. By declaring $x, y ::= @T \mid s$, signatures *are* names. There is nothing to translate. The implementation maintains a single namespace in which quoted signed terms and signatures coexist. This has direct consequences for the Rholang runtime:

- The tuplespace (RSpace) stores both data channels ($@T$) and signature channels (s) in the same structure.
- Pattern matching in for-comprehensions can match on both kinds of name, enabling contracts that dispatch on the kind of authority presented.
- Namespace isolation is enforced by the existing namespace logic [1], now extended to cover the signature sort.

4.2 Cryptographic Quoting

The operator $\#P$ introduces a second axis of reflection. If structural quoting $@T$ is the "mirror" that reflects a signed term into a name preserving all structure (including signature and cost provenance), then cryptographic quoting $\#P$ is a "one-way mirror": it produces a name (usable as a signature, an address, a key) from which the original process cannot be recovered.

This distinction is fundamental. A structurally quoted signed term can be dequoted ($*(@T) = T$) and the process within it re-executed. A cryptographically quoted process cannot: $\#P$ is a commitment, not a capability. This makes $\#P$ suitable for authentication, content addressing, and integrity verification — all operations that require binding to content without granting the power to execute it.

In the F1R3FLY platform, $\#P$ will be implemented as a configurable hash function (SHA-256, Blake2b, etc.) applied to the serialized process tree. The parameterization of s by G ensures that the choice of hash function is a deployment decision, not a calculus-level commitment.

4.3 Valuable Friction at the Syntactic Level

The F1R3FLY platform's design philosophy of *valuable friction* holds that every resource-consuming operation should carry an explicit cost, and that this cost is not merely a runtime check but a *structural feature* of the language. The revised calculus embodies this principle:

- A for-comprehension $\mathbf{for}(y \leftarrow x)\, T$ cannot fire unless its continuation T includes (possibly nested) signed terms with matching tokens.
- A send $x!(U)$ carries a signed term U as payload, so the receiver inherits the sender's cost commitments.
- There is no way to write a communicating process that "escapes" the cost layer, because the grammar does not allow bare processes in the positions where communication occurs.

4.4 Token Stacks as First-Class Terms

Token stacks $S ::= () \mid s : S$ are lists of signatures, representing layered balances. Their inclusion in the signed-term grammar ($T ::= \cdots \mid S$) means that tokens are not inert metadata — they are terms that appear in parallel compositions, flow through channels as parts of signed messages, and participate in reduction.

This design enables several patterns:

- **Token delegation:** A contract can receive a token stack on a channel and use it to fund subsequent communications.
- **Token splitting:** A stack $s_1 : s_2 : S$ can be destructured by separate rules consuming s_1 and s_2 independently.
- **Token markets:** Because tokens are terms, they can be exchanged, auctioned, and composed using standard Rholang contract patterns.

4.5 Genericity over the Cryptographic Backend

The signature grammar is parameterized: $s(G) ::= g \mid \#P \mid s * s$, where $g \in G$ and G is a type parameter. This makes the entire calculus — and therefore the Rholang implementation — generic over the cryptographic backend.

Instantiating G yields a concrete calculus:

- $G = \text{Ed25519}$: ground signatures are Ed25519 public keys.
- $G = \text{secp256k1}$: ground signatures are Ethereum-compatible key hashes.
- $G = \text{Ed25519} + \text{secp256k1}$: a coproduct backend supporting both schemes.

The $*$ operator composes signatures regardless of their ground type, enabling cross-scheme multi-sig patterns. The implementation must provide a ground-signature equality predicate and a hash function for each instantiation of G; the calculus itself is agnostic.

4.6 From Phlogiston to Spectrum: The Decomposition of Cost

In the original Rholang architecture, *phlogiston* was a singular, undifferentiated resource: every deploy consumed units of the same universal fuel, regardless of who signed it or what it computed. The cost-accounted calculus decomposes this singular resource into a *spectrum* of signature-indexed resources — as many distinct fuels as there are economically empowered actors.

Two metaphors illuminate this decomposition:

The prism. Phlogiston is white light. The signature algebra acts as a prism, splitting the undifferentiated beam into its many constituent frequencies. Each signature s indexes a distinct spectral line — a resource pool visible only to processes signed by s. Compound signatures $s_1 * s_2$ produce interference patterns: new frequencies that emerge only when two spectral components are superposed. The linear resource proof at deployment-acceptance time is a *spectral analysis*: it verifies that every frequency the deployment requires is present with sufficient intensity.

Algorithmic chemistry. Alternatively, one may view the calculus as an *algorithmic chemistry* in which economic agency is the catalyst for computation. Processes are reagents; tokens are catalytic substrates; and the rewrite rules are reaction rules that fire only when the appropriate catalyst is present. A signed process $\{P\}_s$ is a reaction waiting for the catalyst s. A token stack $s : S$ is a quantity of catalytic substrate. Communication is a catalyzed reaction: the substrate is consumed, the reagents are transformed, and the products (the continuation plus the remaining substrate) are released.

In this reading, the diversity of signatures corresponds to the diversity of catalysts in a chemical system. Different reactions require different catalysts. A compound signature $s_1 * s_2$ is a *co-catalyst*: both substances must be present for the reaction to proceed. The economy of the F1R3FLY platform is an algorithmic chemistry in which the diversity and abundance of catalysts determine the rate and variety of computation.

Remark 4.1 (Phlogiston as a limit case)**.** *The original phlogiston model is recovered by assigning all actors the same signature s_0. In the prism metaphor, this collapses the spectrum back to white light. In the chemistry metaphor, this reduces the system to a single universal catalyst. The cost-accounted calculus generalizes from this monochromatic / mono-catalytic limit to a full spectrum of economic agency.*

4.7 Funding Slots via Unforgeable Channels

The linear-transfer sugar $\{\mathbf{for}(y \leftarrow x)\, P\}_{s_1 \multimap s_2}$ requires the continuation's funder s_2 to be named at compile time. In practice, one often wants to write a contract whose continuation can be funded by *any* party willing to pay — without knowing their identity in advance. Adding existential quantification over signatures would achieve this but at the cost of complicating the linear resource proofs (the demand function Δ_s would need to range over unknown signatures).

A simpler and more natural solution exploits a feature that Rholang already provides: *unforgeable channels* via **new**. Rather than existentially quantifying over the funder's signature, a contract creates a fresh funding channel — a *funding slot* — and listens for tokens on it. Anyone who knows the channel can deposit tokens there. The linear proof at acceptance time checks whether tokens are present on the funding slot, not who deposited them.

Definition 4.2 (Funding slot)**.** *A funding slot is a pattern in which a contract creates an unforgeable channel and uses it as the signature for a continuation:*

$$\mathbf{new}\ \mathit{slot}\ \mathbf{in}\ \big(\{\mathbf{for}(y \leftarrow x)\, P\}_{s_1 \multimap \mathit{slot}}\ \mid\ @\mathit{slot}!((\cdots))\big)$$

where the send on @slot publishes the funding-slot address to potential funders.

The funding slot decouples three concerns:

Authorization (who may initiate).
Controlled by s_1: only the signer of s_1 can trigger the communication.

Funding (who pays for the continuation).
Open: anyone who deposits tokens on *slot*.

Execution (what happens).
Determined by P: the continuation runs once the slot is funded.

Example 4.3 (Sponsored execution)**.** *A new user u wants to run a deployment D but holds no tokens. A sponsor σ is willing to fund the execution. The on-boarding contract creates a funding slot:*

$$\textbf{new } \mathit{slot} \textbf{ in } \big(\{\textbf{for}(\mathit{dep} \leftarrow u)\, D\}_{u \multimap \mathit{slot}} \mid @\mathit{slot}!(\mathit{slot} : \mathit{slot} : ()) \big)$$

The sponsor funds the slot by depositing tokens:

$$\Sigma[\![\mathit{slot}]\!]!(\mathit{slot} : \mathit{slot} : ())$$

The user u authorizes and triggers the communication (consuming u-tokens); the sponsor's tokens on the funding slot pay for the continuation. The linear proof sees concrete tokens on a concrete channel — no existential quantification is needed.

When the slush fund is exhausted, the user must either attract another sponsor or acquire tokens of their own. The transition from subsidized to self-funded participation is a resource-exhaustion event, not a policy decision.

Remark 4.4 (Unforgeability and security)**.** *The* **new** *operator guarantees that the funding slot is* unforgeable*: only processes that have been explicitly given the channel name can deposit tokens on it. This prevents unauthorized parties from draining the slot or injecting spurious tokens. The capability-security properties of unforgeable channels, already well-understood in the rho calculus, carry over directly to funding slots.*

5 Relationship to the Pure Rho Calculus

The cost-accounted calculus is a *conservative extension* of the pure rho calculus. Setting $T = \{P\}_{s_0}$ for a distinguished "free" signature s_0 with unbounded token supply, and collapsing the signed-term and process levels, recovers the pure rho calculus exactly:

$$\{\textbf{for}(y \leftarrow x)\, \{P'\}_{s_0} \mid x!(\{Q\}_{s_0})\}_{s_0} \mid s_0 : S \rightsquigarrow \{P'\}_{s_0}\{@\{Q\}_{s_0}/y\} \mid S$$

With $s_0 : S$ always available (supplied by a persistent infrastructure process), this reduces to the standard Comm rule. The free signature s_0 plays the role of an "always-funded" system account, modeling the behavior of the pure calculus where cost is not a concern.

This embedding also clarifies the implementation path: the Rholang 1.4 runtime need only implement the cost-accounted calculus, and the "free" mode (for testing, development, and system-level contracts) is obtained by fixing s_0 and providing an inexhaustible token supply.

6 Implications for the F1R3FLY Platform

6.1 Rholang 1.4 (MeTTaIL) Integration

MeTTaIL [4], the Rholang 1.4 language, provides means to define languages as triples of types (syntactic categories), equations on types, and rewrites on types. The cost-accounted rho calculus is itself such a triple:

- **Types:** P, T, S, $s(G)$, and the name sort.
- **Equations:** Structural equivalence ($\equiv_S$) at both the process and signed-term levels.
- **Rewrites:** Rules 1–5 and the contextual closure rules.

This means the cost-accounted calculus can be *defined within MeTTaIL as a GSLT* (graph-structured lambda theory), inheriting MeTTaIL's tooling for type checking, bisimulation, and OSLF-generated logics.

6.2 RSpace and the Tuplespace

In the current F1R3FLY architecture, RSpace is a tuple-space storage engine keyed by channel names. The revised calculus requires RSpace to store entries keyed by both $@T$ and s. Since both are names, this is a matter of extending the serialization format to accommodate the signature sort, not a structural change to RSpace.

Token stacks S are stored as values in signature-keyed entries. The five rewrite rules correspond to tuplespace match-and-consume operations that the reducer already performs, extended with a signature-matching predicate.

6.3 Casper CBC Consensus

The cost-accounting rules interact with Casper CBC consensus at the validator level: a proposed block is valid only if every communication it contains is backed by matching tokens. The revised calculus makes this validity check a syntactic predicate: the block's contents must be well-typed in the cost-accounted grammar, with token stacks present for every signed communication.

6.4 Implementation Path

1. **Phase 1:** Extend the Rholang parser and AST to include the signed-term, token-stack, and parameterized signature sorts. Implement the five rewrite rules in the reducer.
2. **Phase 2:** Extend RSpace serialization to handle signature-keyed entries and token-stack values.
3. **Phase 3:** Define the cost-accounted calculus as a GSLT in MeTTaIL, enabling formal verification via OSLF.
4. **Phase 4:** Expose the signature and token APIs to user contracts, enabling composable metering, cross-chain fee markets, and delegated funding patterns.

7 Transactions, Atomicity, and Static Analysis

The cost-accounted calculus reveals a precise correspondence between the for-comprehension and the concept of a *transaction*. This section develops that correspondence, identifies a partial-funding vulnerability in multi-step transactions, and shows that the vulnerability is eliminated by a decidable static analysis at deployment-acceptance time.

7.1 The For-Comprehension as Transaction Primitive

Every for-comprehension $\mathbf{for}(y \leftarrow x)\, T$ in the cost-accounted calculus is a *transaction* in the precise sense that it decomposes into two funded computational actions:

1. **Rendezvous at the channel.** The runtime must locate a matching send $x!(U)$ on channel x. This is a search operation over the tuplespace, and its cost is paid by one token from the signer's stack.
2. **Matching.** The runtime must verify that the pattern y matches the offered payload U and compute the substitution $\{@U/y\}$. This is a pattern-matching and substitution operation, and its cost is paid by a second token.

The *transaction* is exactly what appears between the round braces of the for-comprehension: the binding specification $(y \leftarrow x)$. Everything else — the continuation T, the surrounding signed-term structure, the token stack — is the *context* in which the transaction executes.

Definition 7.1 (Transaction, witness, state update)**.** *Given a signed for-comprehension* $\{\mathbf{for}(y \leftarrow x)\, T\}_s$ *and a matching send* $x!(U)$*:*

Transaction

The binding specification $(y \leftarrow x)$*: the declaration that we seek a value on channel* x *and will bind it to* y*.*

Witness

The substitution $\{@U/y\}$*: the proof that the transaction succeeded. A message was present on* x*, it matched the pattern, and here is the concrete value that was bound.*

State update

The application of the substitution to the continuation, $T\{@U/y\}$*: the computational effect of the transaction on the system state.*

Atomicity

The substitution is either applied or not. There is no intermediate state. The rewrite rule

$$\{\mathbf{for}(y \leftarrow x)\, T \mid x!(U)\}_s \mid s : S \;\rightsquigarrow\; T\{@U/y\} \mid S$$

fires as a single, indivisible step. If the token is absent or the send is missing, nothing happens — the process blocks.

Remark 7.2 (Database atomicity by construction)**.** *It is important to distinguish two levels of atomicity that the community frequently conflates:* database-transaction atomicity *and* financial-transaction atomicity.

Database-transaction atomicity *is the property of a single communication step: the substitution is either applied or not, the token is either consumed or not, the state is either updated or not. In the cost-accounted rho calculus, this is a* theorem *of the operational semantics — the rewrite rules are defined so that a communication either fires completely or does not fire at all. There is no rule that performs a "partial" communication. No locks, two-phase commit, or rollback logs are needed.*

Financial-transaction atomicity *is a different property: a multi-step operation (e.g., a payment decomposed into debit and credit) should either complete entirely or not begin at all. This is* not *guaranteed by the rewrite rules alone — it requires the static analysis developed in Section 7.5 and the deployment-boundary semantics of Section 7.7.*

7.2 The Partial Funding Vulnerability

Single-step transactions are atomic by construction. The vulnerability arises in *multi-step* transactions: a deployment that chains two or more for-comprehensions sequentially may have enough tokens to fund the first step but not the second.

Consider a payment from Alice to Bob, decomposed into two operations:

1. **Debit:** Extract a purse of amount n from Alice's wallet.
2. **Credit:** Deposit the purse into Bob's wallet.

Each operation is a for-comprehension, each wrapped in a signed term $\{\cdots\}_{Alice}$, each consuming tokens from Alice's stack. If Alice's stack has enough tokens for the debit but not the credit, the system reaches an inconsistent intermediate state: funds extracted from Alice's wallet, sitting in a purse, but the deposit into Bob's wallet never fires because the fuel ran out.

This is a *partial funding vulnerability*: the transaction is semantically atomic (it should either complete or not happen) but operationally non-atomic (it can stall midway through).

7.3 Rholang 1.2 Illustration

We illustrate the vulnerability using Rholang 1.2 syntax, where the ?! operator provides synchronous (function-call-like) communication: `x?!(args)` sends on x with a return channel and blocks until the reply. All three code fragments below use the uniform-signing sugar (Definition 3.6): $\{\mathbf{for}(y \leftarrow x)\, P\}_s$ expands to $\{\mathbf{for}(y \leftarrow x)\, \{P\}_s\}_s$, so each signed for-comprehension consumes *two* tokens (one for the communication, one for the continuation).

Debit handler — extracts a purse from Alice's wallet:

```
// Sugar: costs 2 Alice-tokens (comm + continuation)
{ for( amt ← aliceDebit?!( request, ... ) ){
     let ans = extractPurse( request, ... )
     in rtn!( ans )
} }_{ Alice }
```

Credit handler — deposits a purse into Bob's account:

```
// Sugar: costs 2 Alice-tokens (comm + continuation)
{ for( amt ← bobCredit?!( ... ) ){
    new rcpt in
       addPurseAcct( amt, ... )
       in rtn!( rcpt )
} }_{ Alice }
```

Alice's token stack:

```
Alice : Alice : Alice : Alice : Alice : Alice : S
```

Orchestrating deployment — chains the debit and credit:

```
// Sugar: costs 2 Alice-tokens (comm + continuation)
{ for( amt ← aliceDebit?!( request, ... );
       rcpt ← bobCredit?!( amt, ... ) ){
     P
} }_{ Alice }
```

The total token demand of this deployment is:

Signed layer	Comm	Cont.	Total
Orchestrator's $\{\cdots\}_{Alice}$	1	1	2
Debit handler's $\{\cdots\}_{Alice}$	1	1	2
Credit handler's $\{\cdots\}_{Alice}$	1	1	2
Total	**3**	**3**	**6**

If Alice's stack has at least six tokens, all three signed layers (and their continuations) are funded and the transaction completes. If the stack has only four tokens, the orchestrator and the debit handler fire (consuming four tokens), but the credit handler's signed layer finds no token and *blocks*. Funds are stranded in the purse.

7.4 Decomposing the Transaction Cost

The six-token count of Section 7.3 reflects the uniform-signing sugar expansion of three explicit $\{\cdots\}_{Alice}$ layers. However, the ?! operator itself desugars to a send followed by a for-comprehension on the return channel, so each ?! call involves a for-comprehension on *each* side — the caller's and the handler's. These internal for-comprehensions are also under signed scope and therefore also consume tokens.

A full accounting, decomposed into the two computational actions of Section 7.1, is:

For-comprehension	Rendezvous	Matching	Total
Orchestrator: `aliceDebit`?!	1	1	2
Orchestrator: `bobCredit`?!	1	1	2
Debit handler: **for**	1	1	2
Credit handler: **for**	1	1	2
Total	**4**	**4**	**8**

The six-token count from the sugar expansion is a *syntactic* count of explicitly written signed layers; the eight-token count here is the *semantic* count after full desugaring of ?!. The static analysis of Section 7.5 must use the semantic count, since it is the desugared form that the runtime executes.

The token stack must supply *all* of these or *none* of them. Partial supply leads to partial execution, which is the vulnerability.

7.5 Static Analysis as a Linear Resource Proof

The partial funding vulnerability is eliminated by a *static analysis* performed at deployment-acceptance time — the moment the validator's fuel gate fires and the signed deployment enters execution.

Definition 7.3 (Token demand)**.** *The* token demand *of a signed term T with respect to a signature s, written $\Delta_s(T)$, is the total number of $\{\cdots\}_s$ layers in T and in all processes reachable from T via synchronous calls (?!) or dequotation ($*\cdot$). Formally:*

$$\begin{aligned}
\Delta_s(\{P\}_s) &= 1 + \Delta_s(P) \\
\Delta_s(\{P\}_{s'}) &= \Delta_s(P) \qquad \text{for } s' \neq s \\
\Delta_s(\mathbf{for}(y \leftarrow x)\, T) &= \Delta_s(T) \\
\Delta_s(x!(U)) &= \Delta_s(U) \\
\Delta_s(T \mid U) &= \Delta_s(T) + \Delta_s(U) \\
\Delta_s(*x) &= \Delta_s^{resolve}(x)
\end{aligned}$$

where $\Delta_s^{resolve}(x)$ resolves the name x to its definition (if statically known) and computes Δ_s on the result.

Definition 7.4 (Token supply)**.** *The* token supply *of a signature s in a system is the depth of s-indexed token stacks in the ambient parallel composition:*

$$\begin{aligned}
\Sigma_s(s : S) &= 1 + \Sigma_s(S) \\
\Sigma_s(s' : S) &= \Sigma_s(S) \qquad \text{for } s' \neq s \\
\Sigma_s(()) &= 0 \\
\Sigma_s(T \mid U) &= \Sigma_s(T) + \Sigma_s(U)
\end{aligned}$$

Definition 7.5 (Funding proof obligation)**.** *A deployment is* fully funded *if, for every signature s appearing in the deployment:*

$$\Sigma_s \;\geq\; \Delta_s \tag{15}$$

This is a linear resource inequality*: each token is consumed exactly once, and the total supply must meet or exceed the total demand.*

Theorem 7.6 (Decidability of the funding check)**.** *For a deployment whose call graph is statically known (no higher-order channel passing, no recursive dequotation chains), the funding proof obligation* (15) *is decidable in time linear in the size of the deployment's AST.*

Proof sketch. Δ_s is computed by a single pass over the AST, incrementing a counter at each $\{\cdots\}_s$ node. Σ_s is computed by inspecting the token stacks in the ambient parallel composition. The comparison is a single integer inequality. In the presence of higher-order channels or recursive dequotation, the analysis becomes a conservative over-approximation: unresolvable $*x$ terms contribute an "unknown" demand, and the validator rejects unless the supply exceeds the known lower bound plus a configurable safety margin. □

7.6 The Validator's Acceptance Protocol

The static analysis integrates into the validator's acceptance protocol (Appendix B) as follows:

1. The validator receives a signed deployment $\{D\}_c$ and looks up the client's token stack.
2. **Before** executing any part of the deployment, the validator computes $\Delta_c(D)$ by static analysis of the deployment's AST and call graph.
3. The validator computes Σ_c from the available token stack.
4. If $\Sigma_c \geq \Delta_c$, the deployment is *accepted*: the validator knows, by the linear resource proof, that every for-comprehension in the deployment's execution will be funded. Every substitution that begins will complete. No funds will be stranded.
5. If $\Sigma_c < \Delta_c$, the deployment is *rejected*: the validator returns an error ("insufficient phlogiston") without executing any part of the deployment. No state change occurs. No tokens are consumed.

This protocol converts the database-level atomicity-by-construction of individual rewrite rules into *financial-level* atomicity of multi-step transactions: either the entire transaction is funded and executes to completion, or it is rejected before any step fires. The gap between "each step is atomic" and "the whole transaction is atomic" is bridged by the static analysis at acceptance time.

7.7 Deployment Boundaries and Financial Atomicity

The *deployment* is the unit of financial-transaction atomicity. A deployment is a signed term submitted to a validator for execution. The validator's acceptance protocol (Section 7.6) either accepts the entire deployment (the linear resource proof succeeds) or rejects it entirely (the proof fails). No partial acceptance is possible.

This deployment-boundary semantics resolves several scenarios that are problematic in conventional transaction processing:

Duplicate deployments. Suppose two copies of the same Alice → Bob payment deployment arrive at a validator, and Alice's token stack contains exactly three tokens — enough for one payment but not two. The validator processes each deployment as a separate acceptance decision:

1. The first deployment arrives. The validator computes $\Delta_{Alice} = 3$ and $\Sigma_{Alice} = 3$. The linear proof succeeds: $3 \geq 3$. The deployment is accepted. Three tokens are committed.
2. The second deployment arrives. The validator computes $\Delta_{Alice} = 3$ and $\Sigma_{Alice} = 0$ (the tokens are already committed to the first deployment). The linear proof fails: $0 < 3$. The deployment is rejected. No state change occurs.

The second deployment is rejected *before any step executes*. There is no risk of a partial debit, no stranded purse, no inconsistent state. The linear resource proof ensures that accepted deployments run to completion and rejected deployments have no effect.

Simultaneous arrival. If the two deployments arrive at the same time, the validator treats them as a *single deployment* consisting of the parallel composition of both terms:

$$\{D_1\}_{Alice} \mid \{D_2\}_{Alice}$$

The static analysis computes $\Delta_{Alice} = 3 + 3 = 6$ but $\Sigma_{Alice} = 3$. The linear proof fails: $3 < 6$. The combined deployment is rejected. Neither payment executes.

This is the correct behavior: there are not enough resources to complete both financial transactions, so neither should begin. The linear resource proof detects this automatically, without locks, priority queues, or rollback mechanisms.

Interleaved racing execution. In a naïve runtime that interleaves execution steps from multiple deployments without checking resource bounds, the following race is possible: deployment 1's debit fires (consuming 2 tokens), then deployment 2's debit fires (consuming the last token), and now neither deployment has enough tokens for its credit step. Both payments stall with funds stranded in purses.

The linear resource proof eliminates this race entirely. Acceptance is a *gate*: a deployment passes through the gate only if it carries a proof that all its resources are available. Once accepted, those resources are committed and unavailable to other deployments. There is no interleaving of acceptance and execution — acceptance is a single, atomic decision that precedes all execution.

Remark 7.7 (Deployment acceptance as linear proof search)**.** *The semantics is precise: a deployment is accepted* if and only if *a linear proof of its resource usage exists. The tokens consumed by the proof are* linearly *committed — they cannot be used by any other proof. Two deployments competing for the same tokens are two proof obligations competing for the same linear hypotheses. At most one can succeed. This is not an implementation choice or a heuristic — it is a theorem of linear logic.*

Remark 7.8 (Connection to linear logic)**.** *The funding proof obligation* (15) *is a judgment in the resource-sensitive fragment of linear logic. Each token is a* linear resource *that must be consumed exactly once. The demand Δ_s is the number of linear hypotheses required; the supply Σ_s is the number of linear hypotheses available. A successful funding check is a* proof *in the linear resource logic that the deployment is well-resourced.*

This connection is not merely an analogy. The OSLF functor (observer-generated logic) associated with the cost-accounted GSLT can be extended to generate a linear resource logic whose judgments are *funding proofs. The static analysis is then a proof search in this logic, and the validator is a proof checker.*

8 Conclusion

We have presented a revision of the rho calculus in which cost accounting is folded directly into the grammar and operational semantics, eliminating the need for an external extension or a translation layer. The central design moves are: signatures as names, cryptographic quoting as a second axis of reflection, signed terms pervading the syntax, token stacks as first-class terms, parametric polymorphism of parallel composition, and genericity over the cryptographic backend.

The identification of the for-comprehension as the transaction primitive — with rendezvous and matching as the two funded actions, the substitution as the witness, and application to the continuation as the state update — yields a precise theory of atomicity at two levels. *Database-transaction atomicity* (individual communications) is a theorem of the rewrite rules. *Financial-transaction atomicity* (multi-step operations like payments) is guaranteed by a decidable static analysis at deployment-acceptance time: a linear resource proof that verifies every for-comprehension in the deployment's call graph is funded before any step executes. Deployment boundaries provide the unit of financial atomicity, and the linear proof serves as the concurrency control mechanism — competing deployments are competing proof obligations over the same linear resources, and at most one can succeed.

The result is a calculus that embodies *valuable friction* at the syntactic level: every communicating process must engage the cost layer, and the cost layer is rich enough to guarantee transactional integrity. Because the rho calculus is the formal core of Rholang, this calculus serves directly as the design specification for rearchitecting phlogiston accounting in Rholang 1.4 and the F1R3FLY platform.

E Pluribus Potentia

A Source-to-Source Translation to Pure Rho Calculus

This appendix defines a compositional source-to-source translation from the cost-accounted rho calculus of Section 3 to the *pure* rho calculus. The translation is intended as a small-step development path: one implements it as a compiler pass, and the existing Rholang runtime handles the translated source without modification.

The translation is organized into five mutually recursive functions:

Function	Domain	Codomain	Purpose
$\mathcal{P}[\![\cdot]\!]$	Processes	Pure processes	Translate processes
$\mathcal{N}[\![\cdot]\!]$	Names	Pure names	Translate names
$\mathcal{T}[\![\cdot]\!]$	Signed terms	Pure processes	Translate signed terms
$\mathcal{K}[\![\cdot]\!]$	Token stacks	Pure processes	Translate token stacks
$\Sigma[\![\cdot]\!]$	Signatures	Pure names	Translate signatures

A.1 Signature Translation: $\Sigma[\![\cdot]\!]$

Signatures are mapped to names in the pure rho calculus. Each signature becomes a channel on which fuel tokens will be communicated.

$$\Sigma[\![g]\!] = @H_g \quad \text{for a canonical process } H_g \text{ encoding ground signature } g \in G \tag{16}$$

$$\Sigma[\![\#P]\!] = @H(\mathcal{P}[\![P]\!]) \quad \text{where } H(\cdot) \text{ is a hash-encoding process constructor} \tag{17}$$

$$\Sigma[\![s_1 * s_2]\!] = @\big(*\Sigma[\![s_1]\!] \mid *\Sigma[\![s_2]\!]\big) \tag{18}$$

Equation (18) exploits reflection: the name for a compound signature is the quote of the parallel composition of the dequotations of the component names, ensuring injectivity.

Remark A.1 (Ground-signature encoding)**.** *The canonical process H_g must be injective in g: distinct ground signatures yield distinct processes. In practice, H_g can be a process whose serialized form is the byte sequence of the public key or key hash g. Similarly, $H(\cdot)$ encodes the output of the cryptographic hash function applied to the serialized process tree.*

A.2 Name Translation: $\mathcal{N}[\![\cdot]\!]$

$$\mathcal{N}[\![@T]\!] = @\mathcal{T}[\![T]\!] \tag{19}$$

$$\mathcal{N}[\![s]\!] = \Sigma[\![s]\!] \tag{20}$$

Quoted signed terms become quoted translations. Signatures become their signature-channel names.

A.3 Token-Stack Translation: $\mathcal{K}[\![\cdot]\!]$

A token stack becomes a chain of sends, one per layer, on the corresponding signature channels. Each send carries the translation of the remaining stack as its payload.

$$\mathcal{K}[\![()]\!] = \mathbf{0} \tag{21}$$

$$\mathcal{K}[\![s : S]\!] = \Sigma[\![s]\!]!(\mathcal{K}[\![S]\!]) \tag{22}$$

The token $s : S$ becomes a message on channel $\Sigma[\![s]\!]$ whose payload is the code of the remaining balance $\mathcal{K}[\![S]\!]$. This is the *fuel* that a signed process must consume before it can act.

A.4 Signed-Term Translation: $\mathcal{T}[\![\cdot]\!]$

Signed terms translate to pure processes.

$$\mathcal{T}[\![T \mid U]\!] = \mathcal{T}[\![T]\!] \mid \mathcal{T}[\![U]\!] \tag{23}$$

$$\mathcal{T}[\![S]\!] = \mathcal{K}[\![S]\!] \tag{24}$$

For signed processes $\{P\}_s$, the translation wraps the process in a *fuel gate*: a for-comprehension on the signature channel that must receive a token before the process can proceed. The consumed token's payload (the remaining balance) is released into the continuation via dequotation.

Atomic signature:

$$\mathcal{T}[\![\{P\}_s]\!] \;=\; \mathbf{for}(t \leftarrow \Sigma[\![s]\!])\,\big(*t \mid \mathcal{P}[\![P]\!]\big) \tag{25}$$

The process $\mathcal{P}[\![P]\!]$ is blocked behind the fuel gate $\mathbf{for}(t \leftarrow \Sigma[\![s]\!]) \cdots$. When a token arrives, t is bound to the code of the remaining balance; $*t$ releases it; and the translated process runs in parallel.

Compound signature:

$$\mathcal{T}[\![\{P\}_{s_1 * s_2}]\!] \;=\; \mathbf{for}(t_1 \leftarrow \Sigma[\![s_1]\!])\,\mathbf{for}(t_2 \leftarrow \Sigma[\![s_2]\!])\,\big(*t_1 \mid *t_2 \mid \mathcal{P}[\![P]\!]\big) \tag{26}$$

Both component channels must supply fuel. This generalizes to n-ary composition $s_1 * s_2 * \cdots * s_n$ by nesting n fuel gates.

A.5 Process Translation: $\mathcal{P}[\![\cdot]\!]$

$$\mathcal{P}[\![\mathbf{0}]\!] = \mathbf{0} \tag{27}$$

$$\mathcal{P}[\![\mathbf{for}(y \leftarrow x)\,T]\!] = \mathbf{for}(y \leftarrow \mathcal{N}[\![x]\!])\,\mathcal{T}[\![T]\!] \tag{28}$$

$$\mathcal{P}[\![x!(U)]\!] = \mathcal{N}[\![x]\!]!(\mathcal{T}[\![U]\!]) \tag{29}$$

$$\mathcal{P}[\![P \mid Q]\!] = \mathcal{P}[\![P]\!] \mid \mathcal{P}[\![Q]\!] \tag{30}$$

$$\mathcal{P}[\![*x]\!] = *\mathcal{N}[\![x]\!] \tag{31}$$

A.6 Infrastructure Processes

When the granularity of token holding (combined vs. split) does not match what the fuel gates expect, infrastructure processes mediate the conversion. These are deployed as persistent system-level contracts.

Definition A.2 (Splitter). *Converts a combined token for $s_1 * s_2$ into separate tokens for s_1 and s_2:*

$$\mathrm{Split}(s_1, s_2) \;=\; \mathbf{for}(t \leftarrow \Sigma[\![s_1 * s_2]\!])\,\big(\Sigma[\![s_1]\!]!(\mathbf{0}) \mid \Sigma[\![s_2]\!]!(*t)\big)$$

Definition A.3 (Joiner). *Converts separate tokens for s_1 and s_2 into a combined token for $s_1 * s_2$:*

$$\mathrm{Join}(s_1, s_2) \;=\; \mathbf{for}(t_1 \leftarrow \Sigma[\![s_1]\!])\,\mathbf{for}(t_2 \leftarrow \Sigma[\![s_2]\!])\,\Sigma[\![s_1 * s_2]\!]!((*t_1 \mid *t_2))$$

Remark A.4 (Persistence). *In production, splitters and joiners must be* replicated *(persistent) processes. The standard rho-calculus encoding of replication via self-referencing through reflection applies:*

$$!\,P \;\triangleq\; \mathbf{for}(y \leftarrow @D)\,(*y \mid P) \mid @D!(D) \qquad \textit{where } D = \mathbf{for}(y \leftarrow @D)\,(*y \mid P)$$

A.7 Verification Sketch

We verify that each rewrite rule of the cost-accounted calculus is faithfully simulated by the translation. Write $\|\cdot\|$ for the system-level translation that applies $\mathcal{T}[\![\cdot]\!]$ to every signed-term component and $\mathcal{K}[\![\cdot]\!]$ to every token stack, composing them in parallel.

Rule 1. $\{\mathbf{for}(y \leftarrow x)\,T \mid x!(U)\}_s \mid s : S \;\rightsquigarrow\; T\{@U/y\} \mid S$

Translation of LHS:

$$\begin{aligned}&\mathbf{for}(t \leftarrow \Sigma[\![s]\!])\,\big(*t \mid \mathbf{for}(y \leftarrow \mathcal{N}[\![x]\!])\,\mathcal{T}[\![T]\!] \mid \mathcal{N}[\![x]\!]!(\mathcal{T}[\![U]\!])\big)\\ &\mid\; \Sigma[\![s]\!]!(\mathcal{K}[\![S]\!])\end{aligned}$$

Step 1 — fuel consumption on $\Sigma[\![s]\!]$, binding $t \mapsto @\mathcal{K}[\![S]\!]$:

$$\mathcal{K}[\![S]\!] \mid \mathbf{for}(y \leftarrow \mathcal{N}[\![x]\!])\,\mathcal{T}[\![T]\!] \mid \mathcal{N}[\![x]\!]!(\mathcal{T}[\![U]\!])$$

Step 2 — communication on $\mathcal{N}[\![x]\!]$, binding $y \mapsto @\mathcal{T}[\![U]\!]$:

$$\mathcal{K}[\![S]\!] \mid \mathcal{T}[\![T]\!]\{@\mathcal{T}[\![U]\!]/y\}$$

By compositionality of the translation (the translation commutes with substitution, Proposition A.5), this equals $\mathcal{T}[\![T\{@U/y\}]\!] \mid \mathcal{K}[\![S]\!] = \|T\{@U/y\} \mid S\|$. ✓

Rule 2. $\{\mathbf{for}(y \leftarrow x)\,T \mid x!(U)\}_{s_1 * s_2} \mid s_1 : S \mid s_2 : S' \;\rightsquigarrow\; T\{@U/y\} \mid S \mid S'$

The compound-signature fuel gate (26) nests two **for**-comprehensions on $\Sigma[\![s_1]\!]$ and $\Sigma[\![s_2]\!]$. The two token translations $\mathcal{K}[\![s_1 : S]\!]$ and $\mathcal{K}[\![s_2 : S']\!]$ supply fuel to each. After both fire and the data communication on $\mathcal{N}[\![x]\!]$ completes, the result is $\mathcal{K}[\![S]\!] | \mathcal{K}[\![S']\!] | \mathcal{T}[\![T\{@U/y\}]\!] = \|T\{@U/y\} | S | S'\|$. ✓

Rule 3. $\{\mathbf{for}(y \leftarrow x)\,T \mid x!(U)\}_{s_1 * s_2} \mid (s_1 * s_2) : S \;\rightsquigarrow\; T\{@U/y\} \mid S$

The fuel gate listens on $\Sigma[\![s_1]\!]$ and $\Sigma[\![s_2]\!]$, but the token lives on $\Sigma[\![s_1 * s_2]\!]$. The splitter (Definition A.2) mediates:

$$\mathrm{Split}(s_1, s_2) \mid \Sigma[\![s_1 * s_2]\!]!(\mathcal{K}[\![S]\!]) \;\rightsquigarrow\; \Sigma[\![s_1]\!]!(\mathbf{0}) \mid \Sigma[\![s_2]\!]!(\mathcal{K}[\![S]\!])$$

Now both fuel channels are populated and the reduction proceeds as in Rule 2. The residual $\mathbf{0}$ from the s_1-token is absorbed by structural equivalence. ✓

Rule 4. $\{\mathbf{for}(y \leftarrow x)\,T\}_{s_1} \mid \{x!(U)\}_{s_2} \mid s_1 : S \mid s_2 : S' \;\rightsquigarrow\; T\{@U/y\} \mid S \mid S'$

Each signed process has an atomic fuel gate. The separately signed for-comprehension listens on $\Sigma[\![s_1]\!]$; the separately signed send listens on $\Sigma[\![s_2]\!]$. Both tokens supply fuel independently. After fuel consumption:

$$\mathcal{K}[\![S]\!] \mid \mathbf{for}(y \leftarrow \mathcal{N}[\![x]\!])\, \mathcal{T}[\![T]\!] \mid \mathcal{N}[\![x]\!]!(\mathcal{T}[\![U]\!]) \mid \mathcal{K}[\![S']\!]$$

The data communication on $\mathcal{N}[\![x]\!]$ completes, yielding $\|T\{@U/y\} \mid S \mid S'\|$. ✓

Rule 5. $\{\mathbf{for}(y \leftarrow x)\, T\}_{s_1} \mid \{x!(U)\}_{s_2} \mid (s_1 * s_2) : S \rightsquigarrow T\{@U/y\} \mid S$

The fuel gates listen on $\Sigma[\![s_1]\!]$ and $\Sigma[\![s_2]\!]$ separately, but the token is combined on $\Sigma[\![s_1 * s_2]\!]$. The splitter mediates exactly as in Rule 3, producing separate tokens. After fuel consumption and data communication, the result is $\|T\{@U/y\} \mid S\|$. ✓

A.8 Compositionality

The correctness of the verification rests on the following property:

Proposition A.5 (Translation commutes with substitution)**.** *For all signed terms T, U and variables y:*

$$\mathcal{T}[\![T]\!]\{@\mathcal{T}[\![U]\!]/y\} \;=\; \mathcal{T}[\![T\{@U/y\}]\!]$$

The proof proceeds by structural induction on T, with the base case at $*x$ using the semantic substitution mechanism of the rho calculus. The critical case is:

$$*\mathcal{N}[\![x]\!]\{@\mathcal{T}[\![U]\!]/y\} \;=\; \begin{cases} \mathcal{T}[\![U]\!] & \text{if } y \equiv_N \mathcal{N}[\![x]\!] \\ *\mathcal{N}[\![x]\!] & \text{otherwise} \end{cases}$$

which matches $\mathcal{P}[\![(*x)\{@U/y\}]\!]$ by definition of semantic substitution.

A.9 Implementation Notes

The translation is designed as a **compiler pass** in the Rholang toolchain:

1. **Parse** the extended syntax (processes, signed terms, token stacks, parameterized signatures) into an extended AST.
2. **Apply** $\mathcal{P}[\![\cdot]\!]$ to the top-level process, which recursively invokes $\mathcal{T}[\![\cdot]\!]$, $\mathcal{K}[\![\cdot]\!]$, $\mathcal{N}[\![\cdot]\!]$, and $\Sigma[\![\cdot]\!]$.
3. **Inject** persistent Split and Join infrastructure for every compound signature that appears in the source, unless the programmer has explicitly provided them.
4. **Emit** pure rho calculus (Rholang 1.1) source. The existing runtime, reducer, and RSpace handle execution without modification.

This gives the platform immediate access to cost-accounted semantics while the native implementation (Section 6) is developed in parallel.

B Worked Example: Validator with Fee Extraction

This appendix develops a complete worked example: a simplified model of a validator node in the F1R3FLY network, expressed first in the cost-accounted rho calculus and then translated to pure rho via the compiler pass of Appendix A. The example exercises the bootstrap problem, token lookup, error handling, and conversion of client tokens into validator fees.

B.1 Scenario and Conventions

Fix a ground signature scheme G. The actors are:

Actor	Signature	Role
Validator	$v \in G$	Accepts and executes deploys
Client	$c \in G$	Submits a signed deploy

Since signatures are names, v and c double as channel names. We introduce five auxiliary channels (pure-rho names):

$@DQ_v$	The validator's *deploy queue*
$@F_v$	The validator's fee-collection channel
$@E$	The error-reporting channel
$@A_c$	The address where the client deposits tokens
$@W_v$	The validator's *phlogiston wallet*

Stake vs. phlogiston. It is important to distinguish the validator's *stake* from its *phlogiston* (phlogistons). The stake is the quantity of v-tokens locked in the staking contract; it determines the validator's weight in Casper CBC consensus and is subject to slashing. Phlogiston is a *renewable supply* drawn from the stake and deposited at the wallet channel $@W_v$. The validator's handler consumes phlogiston from $@W_v$ to process client deployments.

At each *epoch boundary* (when the set of staked validators is updated), the protocol replenishes the validator's phlogiston wallet, provided the validator has followed the rules of the protocol. If the validator commits a slashing offense, the entire stake and any remaining phlogistons are moved to a private unforgeable channel where they are held pending review and adjudication. After adjudication, the tokens may be returned (if the offense was spurious), partially redistributed (as a penalty), or burned. Because the unforgeable channel is private, the tokens are effectively revoked: no process can access them without explicit release by the adjudication contract.

In the example below, we model the phlogiston wallet as a message on $@W_v$ carrying a token stack. The validator handler draws tokens from this wallet rather than directly from stake.

B.2 Token Stack Decomposition

We adopt the following structural equivalence for token stacks, which asserts that tokens are *fungible and decomposable*:

$$s : S \;\equiv_S\; (s : ()) \mid S \qquad \text{(token decomposition)} \tag{32}$$

Read: a multi-layer token stack is structurally equivalent to the parallel composition of a single-token stack and the remaining stack. Repeated application decomposes $s : s : s : ()$ into three parallel single-token stacks $\big(s : ()\big) \mid \big(s : ()\big) \mid \big(s : ()\big)$.

This equivalence is justified operationally: in the source-to-source translation (Appendix A), the token stack $s : s : s : ()$ translates to a *nested* chain of sends on $\Sigma[\![s]\!]$, each carrying the remainder as payload. Decomposition corresponds to the standard property that a communication can peel off one layer, releasing the rest.

B.3 Client-Side Assembly

The client prepares a deployment D (a process containing a communication) and deposits three tokens at address $@A_c$:

$$\boxed{C \;\triangleq\; @A_c!(c : c : c : ()) \;\mid\; @DQ_v!(\{D\}_c)} \tag{33}$$

Here $@A_c!(c : c : c : ())$ deposits the three-token stack at the client's token address, and $@DQ_v!(\{D\}_c)$ sends the signed deployment to the validator's deploy queue $@DQ_v$.

For concreteness, let D be a simple communication:

$$D \triangleq \mathbf{for}(z \leftarrow x)\, T_{app} \mid x!(U_{app}) \tag{34}$$

where x, T_{app}, and U_{app} are the client's application-level channel, continuation, and payload.

B.4 Validator Handler

The validator handler is a process that:

1. receives a signed deployment on the deploy queue $@DQ_v$;
2. looks up the client's tokens at address $@A_c$;
3. if tokens are present, extracts one token as a fee and composes the deployment with the remaining tokens;
4. if no tokens are present, reports an error.

We define the handler using the uniform-signing sugar (Definition 3.6):

$$\boxed{VH \triangleq \{\mathbf{for}(dep \leftarrow @DQ_v)\, \mathbf{for}(tok \leftarrow @A_c)\, FeeExtract\}_v} \tag{35}$$

The single outer $\{\cdot\}_v$ suffices because the uniform-signing sugar (13) nests recursively. Expanding one level: $\{\mathbf{for}(dep \leftarrow @DQ_v)\, \mathbf{for}(tok \leftarrow @A_c)\, FeeExtract\}_v$ becomes $\{\mathbf{for}(dep \leftarrow @DQ_v)\, \{\mathbf{for}(tok \leftarrow @A_c)\, Fee$ and the inner $\{\mathbf{for}(tok \leftarrow @A_c)\, FeeExtract\}_v$ is itself sugar, expanding to $\{\mathbf{for}(tok \leftarrow @A_c)\, \{FeeExtract\}_v\}_v$. The fully desugared form therefore has three $\{\cdot\}_v$ layers, consuming three phlogistons.

The fee-extraction process *FeeExtract* applies token decomposition (32) to the received token stack and splits it into a fee portion and a deployment portion:

$$FeeExtract \triangleq @F_v!(c : ()) \mid *dep \mid (c : c : ()) \tag{36}$$

Here:

- $@F_v!(c : ())$ sends one token (the fee) to the validator's fee-collection channel $@F_v$.
- $*dep$ dequotes the received deployment, recovering the signed term $\{D\}_c$.
- $(c : c : ())$ is the remaining two-token stack, placed in parallel with the deployment so that the cost-accounted rewrite rules can fire.

The token decomposition (32) justifies the split: when the validator receives the three-token stack via $\mathbf{for}(tok \leftarrow @A_c) \cdots$, the bound name *tok* carries $@c : c : c : ()$. Dequoting and decomposing:

$$c : c : c : () \equiv_S (c : ()) \mid (c : c : ())$$

Remark B.1 (Error handling)**.** *If the client fails to deposit tokens, the lookup* $\mathbf{for}(tok \leftarrow @A_c) \cdots$ *simply* blocks*: no message is available on* $@A_c$*, and the fuel gate prevents execution. This is the* default *error mode — the deployment is silently rejected by resource starvation.*

For proactive error reporting, the validator can race the token lookup against a timeout. Using the standard select encoding in the rho calculus (a pair of for-comprehensions on the token channel and a timeout channel, with the first to fire pre-empting the other):

$$\mathbf{for}(tok \leftarrow @A_c)\, \{FeeExtract\}_v \mid \mathbf{for}(\_ \leftarrow @timeout)\, \{@E!(\{ErrNoTokens\}_v)\}_v$$

where a timeout process concurrently sends on $@timeout$ *after a deadline.*

B.5 Bootstrap: The Full System

The validator handler VH is itself a signed process $\{\cdots\}_v$. By the cost-accounted rewrite rules, it cannot execute unless v-tokens are present. These tokens come from the validator's phlogiston wallet $@W_v$, which is replenished at each epoch boundary from the validator's stake.

To connect the handler to the wallet, we wrap it in a for-comprehension that draws phlogistons from $@W_v$:

$$\boxed{VB \triangleq \mathbf{for}(phlo \leftarrow @W_v)\,\big(VH \mid *phlo\big)} \tag{37}$$

When a message carrying a token stack arrives on $@W_v$, the variable $phlo$ is bound to the quoted stack; $*phlo$ dequotes it, releasing the tokens into the ambient parallel composition where they can be consumed by VH's signed layers.

The full bootstrapped system is:

$$\boxed{\Omega \triangleq \underbrace{VB}_{\text{validator bootstrap}} \mid \underbrace{@W_v!(v:v:v:())}_{\text{phlogistons from wallet}} \mid \underbrace{C}_{\text{client assembly}}} \tag{38}$$

Expanding VB and C (with VH in sugared form per (35)):

$$\begin{aligned}\Omega \;=\; & \mathbf{for}(phlo \leftarrow @W_v)\,\big(\{\mathbf{for}(dep \leftarrow @DQ_v)\,\mathbf{for}(tok \leftarrow @A_c)\,FeeExtract\}_v \mid *phlo\big) \\ & \mid\; @W_v!(v:v:v:()) \\ & \mid\; @A_c!(c:c:c:()) \;\mid\; @DQ_v!(\{D\}_c)\end{aligned} \tag{39}$$

Recall that VH uses the recursive uniform-signing sugar (Definition 3.6): the single outer $\{\cdot\}_v$ expands recursively to three $\{\cdot\}_v$ layers in the fully desugared form, one per for-comprehension plus one for $FeeExtract$.

The validator's stake (locked in the staking contract) is *not* consumed directly. The three phlogistons $v:v:v:()$ on $@W_v$ are a renewable budget drawn from stake, matched exactly to the three desugared signed layers of the handler.

B.6 Reduction in the Cost-Accounted Calculus

We trace the reduction of Ω using the cost-accounted rewrite rules of Section 3.6. At each step we identify which rule applies.

Step 0 — The bootstrap process draws execution tokens from the wallet (COMM on $@W_v$):

The for-comprehension $\mathbf{for}(phlo \leftarrow @W_v)\cdots$ communicates with $@W_v!(v:v:v:())$, binding $phlo \mapsto @v:v:v:()$. After dequotation of $*phlo$, the system becomes:

$$\begin{aligned}&\{\mathbf{for}(dep \leftarrow @DQ_v)\,\{\mathbf{for}(tok \leftarrow @A_c)\,\{FeeExtract\}_v\}_v\}_v \;\mid\; v:v:v:() \\ &\mid\; @A_c!(c:c:c:()) \;\mid\; @DQ_v!(\{D\}_c)\end{aligned}$$

The phlogistons are now in the ambient parallel composition, available to the handler's signed layers. Note that this step is *not* a cost-accounted rewrite: the bootstrap for-comprehension on $@W_v$ is an unsigned process (a pure-rho communication), not a signed term. It is free infrastructure, analogous to the operating system's process scheduler.

Step 1 — The outermost signed handler consumes one phlogiston and receives the signed deployment (**Rule 1**):

The signed handler's outermost layer is $\{\mathbf{for}(dep \leftarrow @DQ_v)\cdots\}_v$, and the send $@DQ_v!(\{D\}_c)$ provides a message on $@DQ_v$. With token $v:v:v:()$, decomposed as $(v:()) \mid (v:v:())$:

$$\begin{aligned}&\{\mathbf{for}(dep \leftarrow @DQ_v)\cdots \mid @DQ_v!(\{D\}_c)\}_v \;\mid\; v:() \\ &\quad\rightsquigarrow\quad \cdots\{@\{D\}_c/dep\}\end{aligned}$$

After this step, *dep* is bound to $@\{D\}_c$ and one phlogiston is consumed. The system becomes:

$$\{\mathbf{for}(tok \leftarrow @A_c)\,\{\mathit{FeeExtract}\}_v\}_v \mid v : v : () \\ \mid @A_c!(c : c : c : ())$$

(with $dep = @\{D\}_c$ captured in *FeeExtract*).

Step 2 — The second layer consumes one execution token and looks up the client's tokens (**Rule 1**):

$$\{\mathbf{for}(tok \leftarrow @A_c) \cdots \mid @A_c!(c : c : c : ())\}_v \mid v : () \\ \rightsquigarrow \quad \{\mathit{FeeExtract}\}_v\{@c : c : c : ()/tok\} \mid v : ()$$

Now $tok = @c : c : c : ()$ and one more phlogiston is consumed. One phlogiston remains.

Step 3 — The innermost layer consumes the last phlogiston and executes fee extraction (**Rule 1**):

$$\{\mathit{FeeExtract}\}_v \mid v : () \quad \rightsquigarrow \quad \mathit{FeeExtract} \mid ()$$

Substituting the bound values into *FeeExtract* (36) and applying token decomposition:

$$@F_v!(c : ()) \mid \{D\}_c \mid (c : c : ()) \tag{40}$$

The system now contains the signed deployment $\{D\}_c$ in parallel with two client tokens ($c : c : ()$), plus a fee token ($c : ()$) en route to the validator's fee channel.

Step 4 — The client deployment fires (**Rule 1**):
Expanding $D = \mathbf{for}(z \leftarrow x)\,T_{app} \mid x!(U_{app})$:

$$\{\mathbf{for}(z \leftarrow x)\,T_{app} \mid x!(U_{app})\}_c \mid c : c : () \\ \rightsquigarrow \quad T_{app}\{@U_{app}/z\} \mid c : ()$$

One client token is consumed. The application continuation runs with one remaining client token.

Final state:

$$@F_v!(c : ()) \mid T_{app}\{@U_{app}/z\} \mid (c : ())$$

The fee token is deposited at the validator's fee channel. The application continuation runs. One client token remains for further computation.

B.7 Source-to-Source Translation

We now apply the translation of Appendix A to the system Ω. Write $n_v = \Sigma[\![v]\!]$ and $n_c = \Sigma[\![c]\!]$ for the translated signature channels.

Validator tokens:

$$\mathcal{K}[\![v : v : v : ()]\!] = n_v!(n_v!(n_v!(\mathbf{0})))$$

A triple-nested send on n_v.

Client tokens:

$$\mathcal{K}[\![c : c : c : ()]\!] = n_c!(n_c!(n_c!(\mathbf{0})))$$

Validator handler (outermost fuel gate only, for brevity):

$$\mathcal{T}[\![\,VH]\!] \;=\; \mathbf{for}(t_1 \leftarrow n_v)\,\big(*t_1 \mid \mathbf{for}(\mathit{dep} \leftarrow n_v)\,\mathcal{T}[\![\text{inner}]\!]\big)$$

where $\mathcal{T}[\![\text{inner}]\!]$ nests two more fuel gates on n_v (one per remaining $\{\cdots\}_v$ layer), followed by the token lookup on $@A_c$, the fee extraction, and the translated deployment.

Fee extraction in the translated system:

The critical observation is that fee extraction corresponds to an *additional fuel gate* on the client's signature channel n_c. After the validator handler runs, the translated system contains (among other terms):

$$\underbrace{\mathbf{for}(t_{\mathit{fee}} \leftarrow n_c)\,\big(@F_v!(\mathbf{0}) \mid *t_{\mathit{fee}}\big)}_{\text{fee interceptor}} \;\mid\; \underbrace{n_c!(n_c!(n_c!(\mathbf{0})))}_{\text{client's 3 tokens}} \tag{41}$$

The fee interceptor consumes the outermost send on n_c (one token). Its payload is $n_c!(n_c!(\mathbf{0}))$ — the remaining two tokens — which is released via $*t_{\mathit{fee}}$. The fee is recorded as a send on $@F_v$.

After this reduction:

$$@F_v!(\mathbf{0}) \;\mid\; n_c!(n_c!(\mathbf{0})) \;\mid\; \underbrace{\mathbf{for}(t \leftarrow n_c)\,\big(*t \mid \mathcal{P}[\![D]\!]\big)}_{\text{deployment fuel gate}}$$

The deployment's own fuel gate now consumes from the remaining two-token chain on n_c, exactly as verified in Appendix A.7.

B.8 Reduction Trace in Pure Rho

We trace the key reductions in the translated system, suppressing intermediate structural equivalence steps.

Step 0. Wallet draw on $@W_v$. The bootstrap for-comprehension receives the phlogiston stack. $*\mathit{phlo}$ releases three sends on n_v into the ambient parallel composition.

Step 1. Validator fuel gate 1 fires on n_v. The outermost send in the phlogiston chain is consumed. The handler's first **for** unblocks. Remainder: two sends on n_v.

Step 2. Deployment received on n_v. The translated $n_v!(\mathcal{T}[\![\{D\}_c]\!])$ communicates with the handler's inner **for** on n_v. *dep* is bound.

Step 3. Validator fuel gate 2 fires on n_v. One more phlogiston consumed.

Step 4. Token lookup on $@A_c$. The client's token deposit communicates with the handler's **for** on $@A_c$. *tok* is bound to the quoted three-token chain.

Step 5. Validator fuel gate 3 fires on n_v. Last phlogiston consumed. *FeeExtract* is now active.

Step 6. Fee interceptor fires on n_c (41). One client token consumed as fee. Two-token chain released.

Step 7. Deployment fuel gate fires on n_c. One client token consumed. Application continuation $\mathcal{P}[\![T_{\mathit{app}}]\!]$ unblocks. One-token chain released.

Step 8. Application communication on $\mathcal{N}[\![x]\!]$. The translated $\mathbf{for}(z \leftarrow \mathcal{N}[\![x]\!]) \cdots$ and $\mathcal{N}[\![x]\!]!(\cdots)$ communicate. The application payload is delivered.

Final state (in pure rho):

$$\underbrace{@F_v!(\mathbf{0})}_{\text{fee deposited}} \;\mid\; \underbrace{\mathcal{T}[\![T_{\mathit{app}}\{@U_{\mathit{app}}/z\}]\!]}_{\text{application result}} \;\mid\; \underbrace{n_c!(\mathbf{0})}_{\text{1 client token remaining}}$$

B.9 Discussion

Stake, phlogistons, and epochs. The validator's *stake* — the quantity of v-tokens locked in the staking contract — is not consumed directly by the handler. Instead, the staking contract deposits a renewable supply of *phlogistons* at the wallet channel $@W_v$ at each epoch boundary (the point at which the set of staked validators is updated). The handler draws from this wallet.

The phlogiston wallet must supply at least as many tokens as the handler's nesting depth per deployment. In the example, three $\{\cdots\}_v$ layers require three phlogistons per deploy. A persistent validator ($!\,VB$) re-draws from the wallet after each deployment, consuming phlogistons at a rate proportional to the throughput of client deploys.

Slashing. In the current F1R3FLY slashing algorithm, any slashing offense results in the removal of the validator's entire stake. In the cost-accounted model, slashing is implemented as a *channel revocation*: the slashing contract moves the validator's stake and any remaining phlogistons to a private unforgeable channel, where they are inaccessible to the validator's handler.

Because the handler draws phlogistons from $@W_v$, moving the wallet's contents to a private channel immediately halts the validator: the bootstrap for-comprehension $\mathbf{for}(phlo \leftarrow @W_v)\cdots$ finds no message and blocks. No further deployments can be processed. The validator is effectively offline.

The tokens held at the private channel are not destroyed. They remain available for recovery pending review and adjudication. After adjudication, the tokens may be:

- **Returned** to the validator's wallet (if the offense was spurious or the result of a network fault).
- **Partially redistributed** to affected parties (as a penalty proportional to the severity of the offense).
- **Burned** (removed from circulation, reducing the total token supply).

The adjudication contract is itself a Rholang program, expressible in the cost-accounted calculus, and subject to the same formal verification tools.

Fee conversion. The fee token $(c : ())$ is a *client-signature* token. For the validator to use it as fuel for its own operations (which require v-signature tokens), the fee must be *converted*. This conversion is a market-making operation, expressible as a Rholang contract:

$$\text{Exchange}(c, v) \;=\; \mathbf{for}(t_c \leftarrow n_c)\,\mathbf{for}(t_v \leftarrow n_v)\,\big(n_c!(*t_v) \mid n_v!(*t_c)\big)$$

This persistent exchange contract swaps one c-token for one v-token (and vice versa), implementing a 1:1 peg. More sophisticated exchange contracts can implement variable rates, order books, or automated market makers — all within Rholang.

Persistent validator. The example bootstrap processes a single deployment and halts. A production validator runs persistently, re-drawing phlogistons from the wallet after each deployment. Using the standard reflective replication encoding:

$$!\,VB$$

Each iteration draws phlogistons from $@W_v$, processes a deployment, collects fees, and (via the exchange contract) converts fees to replenish the phlogiston supply — establishing the economic feedback loop that sustains the validator between epoch boundaries.

Connection to Casper CBC. In the F1R3FLY platform, the validator's stake (v-tokens locked in the staking contract) determines the validator's weight in Casper CBC consensus. The cost-accounted calculus makes this relationship *intrinsic*: the validator literally cannot act without phlogistons drawn from stake, and the rate of token consumption is determined by the structure of the handler — the same structure that the consensus layer inspects. Slashing, epoch-boundary replenishment, and stake-weighted voting are all expressible as cost-accounted Rholang contracts, unifying the consensus and execution layers under a single formal semantics.

Acknowledgements

The author thanks Mike Stay for extensive discussions on the connections between linear logic and resource accounting, Dylon Edwards for detailed feedback on the operational semantics and implementation path, and Kevin Machiels for insightful comments on the validator economics and slashing model.

References

[1] L. G. Meredith and M. Radestock, "A reflective higher-order calculus," *Electronic Notes in Theoretical Computer Science*, vol. 141, no. 5, pp. 49–67, 2005.

[2] R. Milner, *Communicating and Mobile Systems: the π-Calculus*, Cambridge University Press, 1999.

[3] L. G. Meredith *et al.*, "Rholang Specification," F1R3FLY.io / RChain Cooperative, 2017–2026.

[4] L. G. Meredith, "MeTTaIL (Rholang 1.4): A language for defining languages," F1R3FLY.io, 2025–2026.

[5] L. G. Meredith *et al.*, "Secure Blockchain-Based Health Data Platform: Cooperative Research and Development Agreement," F1R3FLY Industries / U.S. Department of Veterans Affairs, 2025–2026.